

Value Objects

Modeling the domain with safe types

Carlos Terrero Orpí

basic_crud — Product domain

What is a Value Object?

An **immutable object** defined by **the value of its attributes**, not by an identity.

- It has no `id`
- Two VOs with the same value **are the same VO**
- Self-validated on construction
- Once created, **it never changes**

"If I have two banknotes and both are 5€, are they the same banknote? It doesn't matter: to pay for the coffee, they're interchangeable."

The problem: primitive obsession

```
class Product
{
    private ?string $name = null;

    public function setName(string $name): void
    {
        $this->name = $name;    // empty? whitespace? 2 characters?
    }
}
```

- Any `string` gets through: `""`, `" "`, `"a"`, `"\n"`
- Validation is duplicated in every `setName`, controller, command...
- The `string` type doesn't communicate intent

The solution: **ProductName**

```
namespace App\Product\Entity\ValueObjects;

final readonly class ProductName
{
    private const int MIN_LENGTH = 3;

    public string $value;

    public function __construct(string $value)
    {
        $this->value = trim($value);
        if ($this->value === '') {
            throw ProductNameException::empty();
        }
        if (strlen($this->value) < self::MIN_LENGTH) {
            throw ProductNameException::tooShort($this->value, self::MIN_LENGTH);
        }
    }
}
```

Anatomy: **final readonly**

```
final readonly class ProductName
```

Modifier	Guarantee
final	No one can extend it and break the invariants
readonly	Properties are immutable after construction
public string \$value	Direct access without a ceremonial getter

Validation in the constructor

```
public function __construct(string $value)
{
    $this->value = trim($value);

    if ($this->value === '') {
        throw ProductNameException::empty();
    }
    if (strlen($this->value) < self::MIN_LENGTH) {
        throw ProductNameException::tooShort($this->value, self::MIN_LENGTH);
    }
}
```

Always valid principle: if the constructor finishes without an exception, the object is in a valid state. Always.

Domain exceptions

```
namespace App\Product\Entity\ValueObjects\Exception;

final class ProductNameException extends InvalidArgumentException
{
    public static function empty(): self
    {
        return new self('Product name cannot be empty');
    }

    public static function tooShort(string $value, int $minLength): self
    {
        return new self(
            sprintf('Product name "%s" needs to be at least %d characters or more', $value, $minLength)
        );
    }
}
```

Named constructors → a single type, two well-described ways to fail.

Why custom exceptions?

Generic (<code>InvalidArgumentException</code>)	Domain (<code>ProductNameException</code>)
Any library can throw it	Only our VO throws it
Ambiguous catch	<code>catch (ProductNameException \$e)</code>
Message as a loose string	Centralized in the class
Hard to trace	Speaks the language of the domain

```
try {  
    new ProductName($input);  
} catch (ProductNameException $e) {  
    // I know exactly what failed and where  
}
```


Equality by value

```
public function test_check_valueObject_has_the_same_value(): void
{
    $productNameFirst = new ProductName('Test Product');
    $productNameSecond = new ProductName('Test Product');

    $this->assertEquals($productNameFirst, $productNameSecond);
}
```

- `assertEquals` compares **value** (not reference)
- Two `ProductName('Test Product')` are indistinguishable
- It's the essence of the Value Object

Transparent normalization

```
public function test_that_valueObjects_trim_the_name()
{
    $name = '    Test Product    ';
    $productName = new ProductName($name);

    $this->assertEquals('Test Product', $productName->value);
}
```

The VO **cleans up the input** before validating:

- " Test Product " → "Test Product"
- " " → empty → exception

Whoever constructs it doesn't have to remember to `trim`.

Validation tests

```
public function test_throws_ProductNameException_on_empty_name(): void
{
    $this->expectException(ProductNameException::class);
    $this->expectExceptionMessage('Product name cannot be empty');
    new ProductName('');
}

public function test_rejects_whitespace_only_name(): void
{
    $this->expectException(ProductNameException::class);
    $this->expectExceptionMessage('Product name cannot be empty');
    new ProductName(' ');
}

public function test_throws_ProductNameException_with_less_than_three_chars(): void
{
    $this->expectException(ProductNameException::class);
    $this->expectExceptionMessage('needs to be at least 3 characters');
    new ProductName('as');
}
```

Entity vs Value Object

	Entity (Product)	Value Object (ProductName)
Identity	id (ULID)	By value
Mutability	Yes (setName , setPrice)	Immutable
Equality	By id	By value
Lifecycle	Has a history	Has none
Example	"This specific product"	"The name 'T-shirt'"

Benefits

1. **Type safety**: the compiler / static analyzer rejects invalid uses
2. **Validation in a single place**: the constructor
3. **Explicit intent**: `ProductName > string`
4. **Focused tests**: you test the VO, not every place that uses it
5. **Safe refactoring**: changes to the concept = changes to the VO
6. **Living documentation**: the code describes the domain

But they're not a silver bullet

Everything above comes at a price.

A VO **solves one specific problem**, the scattered validation of a value with invariants, **not every problem**.

Wrapping for the sake of wrapping is *over-engineering* by another name.

Cost 1: boilerplate and proliferation

A single concept = **three files** (VO + exception + tests):

```
ProductName.php  
ProductNameException.php  
ProductNameTest.php
```

- Multiply it by every field: `Price`, `Sku`, `Slug`, `Stock` ...
- More classes = more surface to maintain and navigate
- The `->value` leaks through all the code that consumes the VO

For a value **with no real invariants**, an honest `string` beats a ceremonial VO.

Cost 2: friction at the boundaries

The outside world speaks in **primitives**: JSON, forms, DB columns.

```
// Input: you have to build the VO from the primitive
$name = new ProductName($request->get('name'));

// Persistence (MongoDB / Doctrine ODM):
// you need a custom type or to map it by hand
$document['name'] = $product->name->value;

// Output (API / serialization): back to a string
['name' => $product->name->value]
```

Every edge of the system is a **conversion** ↔ **hydration** point that didn't exist before.

Cost 3: invariants that span fields

A VO only knows **its own value**. It can't validate rules between fields:

```
// ✗ ProductName knows nothing about the price  
// "the discounted price must be < base price"
```

- Those rules live in the **entity** or in a **domain service**
- The VO doesn't free you from having domain logic elsewhere
- Risk: believing "everything is already validated" because each field is a VO

When **not** to use a Value Object

Situation	Better option
Value with no invariants (free note, flag)	Primitive
Data that just passes through (transport DTO)	array / readonly DTO
Prototype / trivial CRUD	Start simple, refactor later
Rule that spans several fields	Entity or domain service
The team doesn't share the DDD language	Weigh the maintenance cost

Rule of thumb

Create a VO when there are **invariants to protect** and the concept **repeats or has its own rules**.

- Is any value of the type valid? → primitive
- Are there rules, normalization or a domain concept? → VO
- When in doubt: start with the primitive and **promote it to a VO when it hurts**

The VO is a tool, not a dogma.

Summary

A Value Object is:

immutable · self-validated · no identity · equal by value

Why?

To model the domain with types that **can't be wrong**.

Questions?

VO: `src/Product/Entity/ValueObjects/ProductName.php`

Exception:

`src/Product/Entity/ValueObjects/Exception/ProductNameException.php`

Tests: `tests/Product/ValueObjects/ProductNameTest.php`